



NORTH AMERICAN
UNIVERSITY

Spotify Songs Dataset

Data Preprocessing
COMP4353(1) — Data Mining

Students: Ester Hasmata, Gonxhe Halimi
Professor: Sabina Adhikari



Objective

- Preprocess a real-world Spotify Songs Dataset for analysis and modeling
- Apply data preprocessing techniques to understand patterns in the data
- Build and evaluate machine learning models on the cleaned dataset

STEP 1

Data Cleaning

```
gonxhehalimi@Gonxhes-MacBook-Air-6 Desktop % python3 data.py
  Artist      Track ... Likes_Normalized Stream_Normalized
0  Coldplay      Yellow ...      0.148189      0.438096
1  Coldplay  Viva La Vida ...      0.140766      0.391427
2  Coldplay      Sparks ...      0.025359      0.140366
3  Coldplay  Something Just Like This ...      0.331183      0.599679
4  Coldplay      The Scientist ...      0.178202      0.432887

[5 rows x 14 columns]

Missing values:
  Artist      0
  Track      0
  Album      0
  Album_type  0
  Genre      0
  Views      0
  Likes      0
  Licensed   0
  Stream     0
  Most_playedon  0
  Popularity_score  0
  Views_Normalized  0
  Likes_Normalized  0
  Stream_Normalized  0
dtype: int64

Data cleaned and saved to Desktop!
```

What We Found

Some numerical values contained commas, causing potential errors. No missing values or noisy data were detected.

Techniques Applied

- Removed commas using `str.replace(",", "")`
- Converted text to numeric with `pd.to_numeric()` and `errors="coerce"`
- Inspected structure via `head()`, `dtypes`, and `describe()`
- Created a working copy using `data.copy()`

STEP 2

Data Reduction

Attribute Subset Selection

Removed less useful features (loudness, speechiness, acousticness, etc.) and kept only key columns:

Views_Normalized, Likes_Normalized, Stream_Normalized, and **Album_type**. Text columns like Artist and Track were excluded.

Sampling

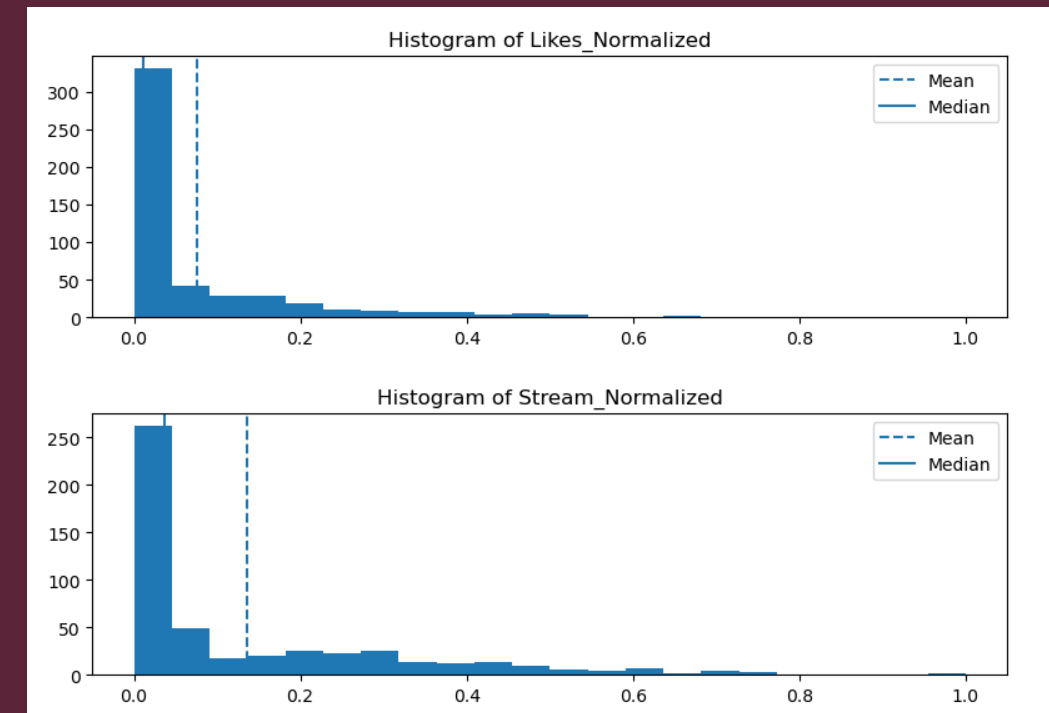
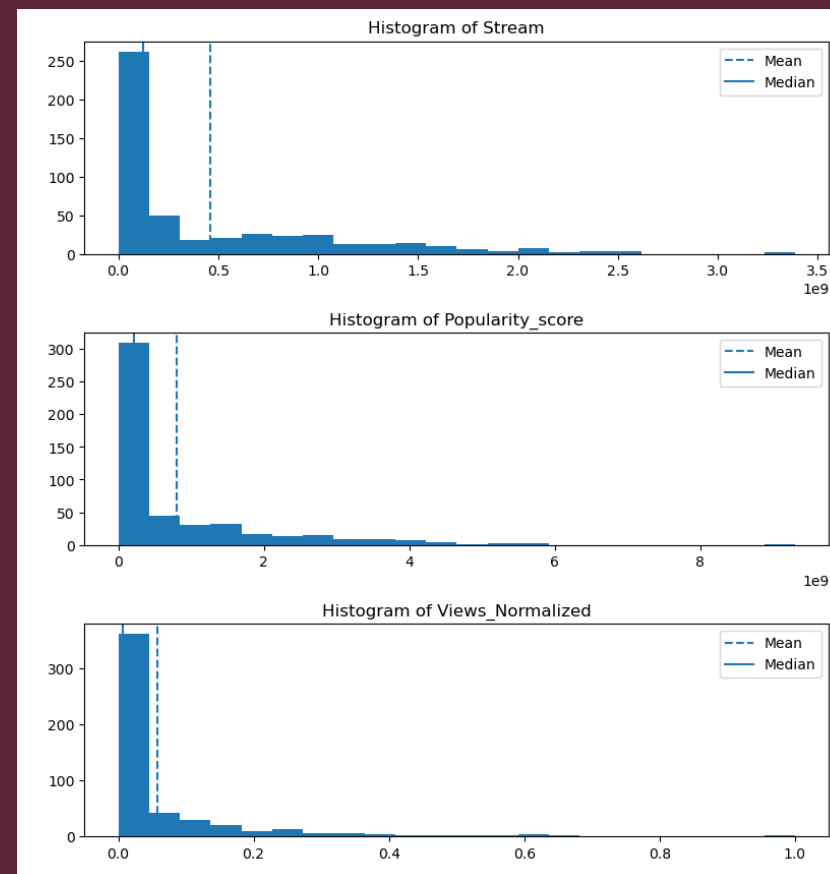
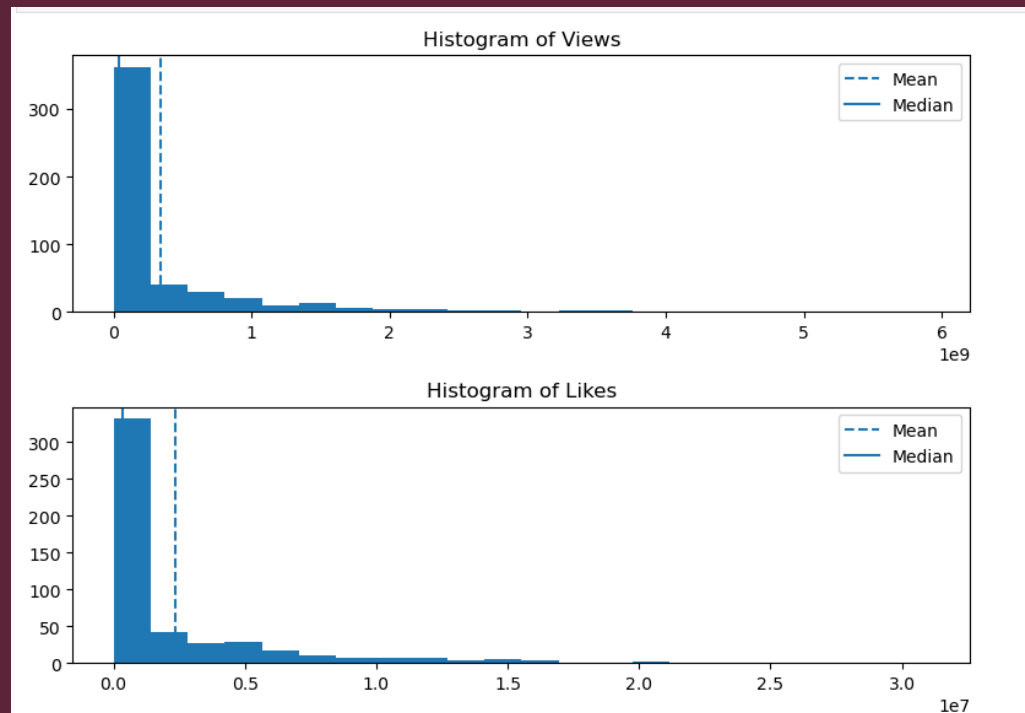
Reduced from 20,000+ songs to a manageable sample of **500 songs**. Split into training and testing subsets using `train_test_split()` from sklearn.

```
[gonxhehalimi@Gonxhes-MacBook-Air-6 Desktop %  
[gonxhehalimi@Gonxhes-MacBook-Air-6 Desktop % python3 data_reduction.py  
Original columns: ['Artist', 'Track', 'Album', 'Album_type', 'Genre', 'Views', 'Likes', 'Licensed', 'Stream', 'Most_playedon', 'Popularity_score', 'Views_Normalized', 'Likes_Normalized', 'Stream_Normalized']  
  
Reduced columns: ['Views_Normalized', 'Likes_Normalized', 'Stream_Normalized', 'Album_type']  
Views_Normalized Likes_Normalized Stream_Normalized Album_type  
0 0.140907 0.148189 0.438096 album  
1 0.133637 0.140766 0.391427 album  
2 0.012389 0.025359 0.140366 album  
3 0.358476 0.331183 0.599679 album  
4 0.183229 0.178202 0.432887 album  
  
Reduced dataset saved as 'spotify_reduced.xlsx'
```

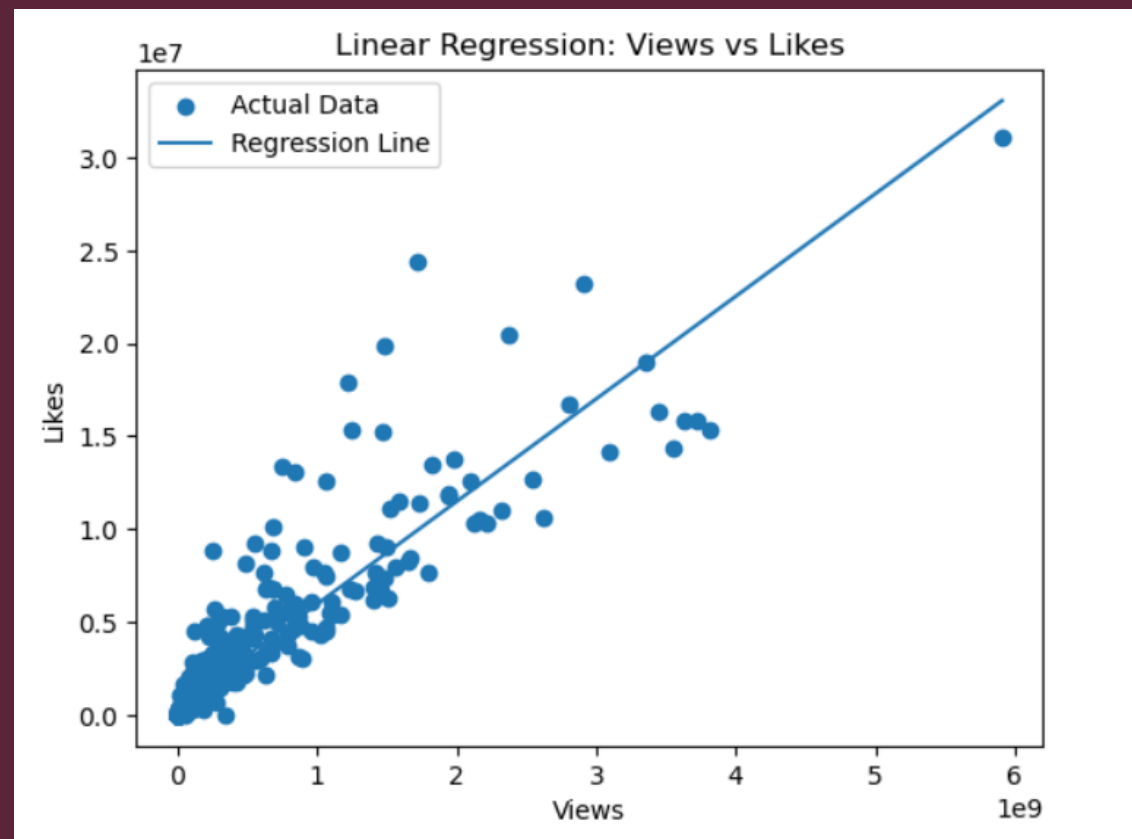
STEP 2 — CONTINUED

Histogram Analysis

We used histogram analysis with *sns.histplot()* to show the distribution of numerical values such as views, likes, streams, which helped us see how the data is spread across different ranges.



Linear Regression



Model Formula

$$\text{Likes} = 0.01 \times \text{Views} + 456,888.48$$

- We built a linear regression model to understand the relationship between Views and Likes.
- The model showed a positive relationship between the two variables.

Data Transformation

Label Encoding

Categorical variables converted to integers for ML compatibility:

- **Licensed:** True/False → 1/0
- **Most_playedon:** Spotify/YouTube → 1/0
- **Album_type:** Album=2, Single=1, Compilation=0
- **Genre:** Encoded into numerical labels

Normalization

The original values for Views, Likes, and Stream are very large, but after normalization they are scaled down to values between 0 and 1, making them easier to compare and ensuring that no single variable dominates the analysis.

Album_type	Genre	Views	Likes	Licensed	Stream	Most_playedon	Popularity_score	Views_Normalized	Likes_Normalized	Stream_Normalized
2	5.0	832532409	4600933	1	1483622308	1	2320755650	0.140907	0.148189	0.438096
2	5.0	789581468	4370461	1	1325575712	1	2119527641	0.133637	0.140766	0.391427
2	5.0	73201019	787343	1	475353044	1	549341406	0.012389	0.025359	0.140366
2	5.0	2118018686	10282499	1	2030825694	0	4159126879	0.358476	0.331183	0.599679
2	5.0	1082588109	5532787	1	1465980600	1	2554101496	0.183229	0.178202	0.432887

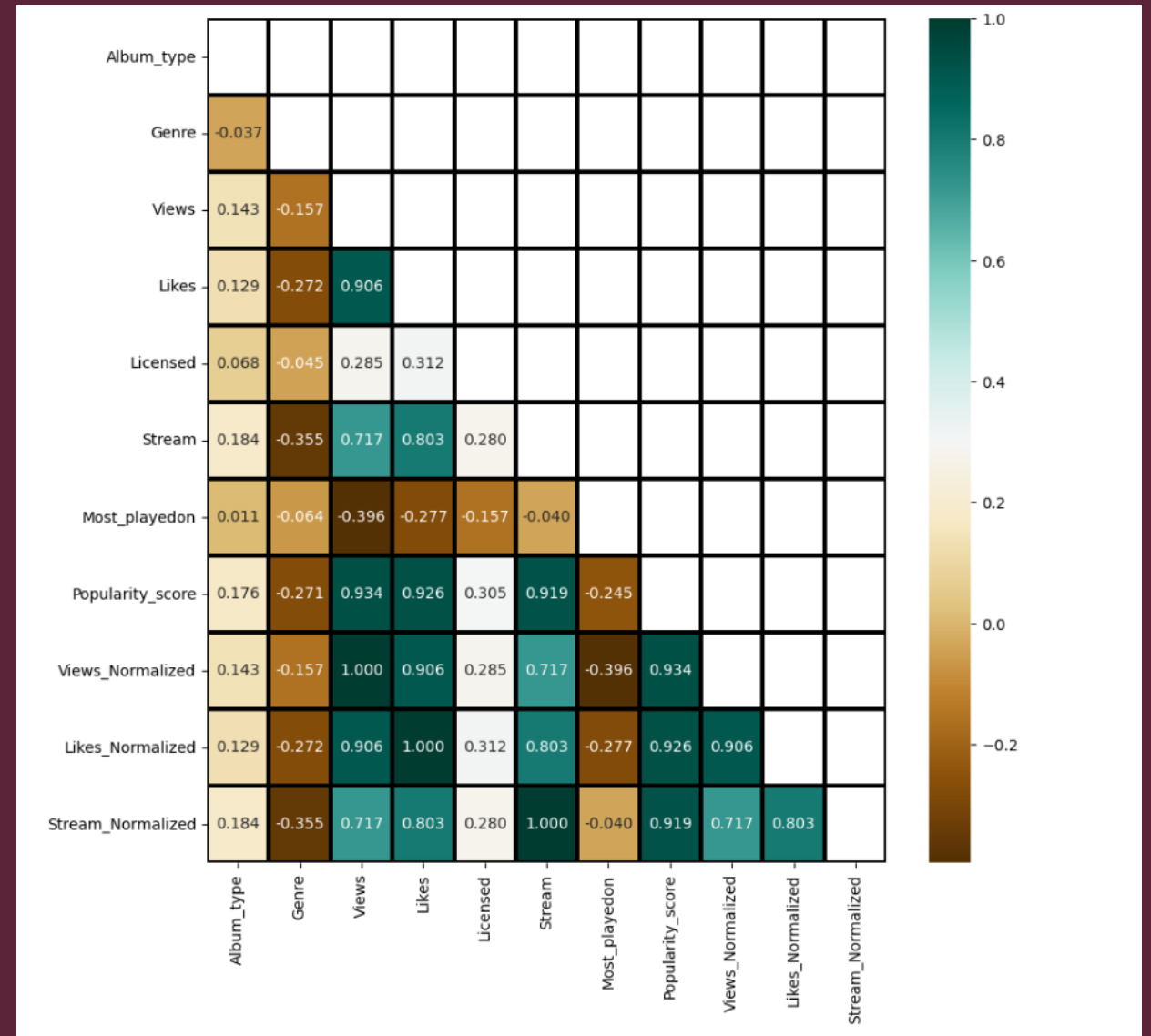
Correlation Analysis

Key Finding

The correlation analysis shows that Views, Likes, and Stream are strongly positively correlated, meaning that songs with higher views tend to also have higher likes and streams.

Correlation Scale

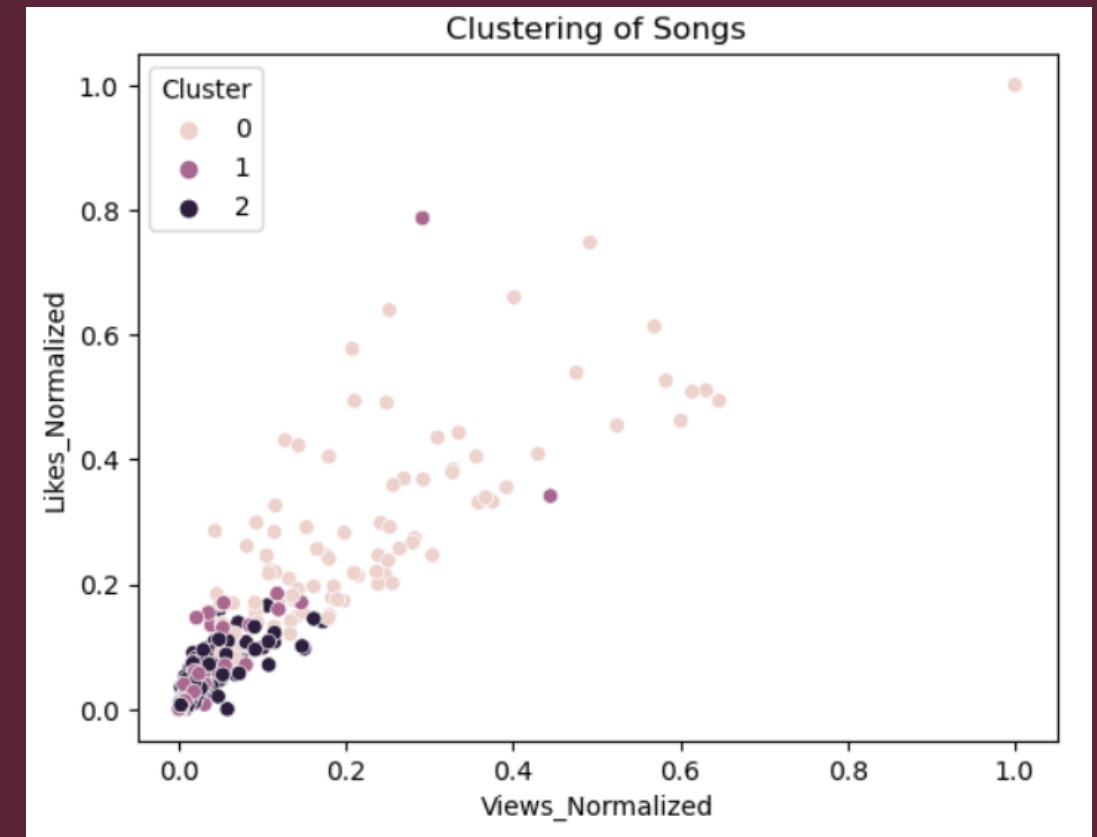
- **-1**: Perfect negative (opposite directions)
- **0**: No correlation
- **+1**: Perfect positive (same direction)



K-Means Clustering

Setup

We applied K-Means clustering with 3 clusters using features like *Views_Normalized*, *Likes_Normalized*, *Stream_Normalized*, and *Album_type*. The model grouped songs into three categories based on their popularity.



Clusters Identified

High Popularity

Medium Popularity

Low Popularity

Classification: Engagement Score

Views	Likes	Licensed	Stream	Most_playedon	Popularity_score	Views_Normalized	Likes_Normalized	Stream_Normalized	Cluster	Engagement_Score	Engagement_Level
832532409	4600933	1	1483622308	1	2320755650	0.140907	0.148189	0.438096	0	1.249100	Medium
789581468	4370461	1	1325575712	1	2119527641	0.133637	0.140766	0.391427	0	1.176844	Medium
73201019	787343	1	475353044	1	549341406	0.012389	0.025359	0.140366	2	4.548889	High
2118018686	10282499	1	2030825694	0	4159126879	0.358476	0.331183	0.599679	0	0.672639	Low
1082588109	5532787	1	1465980600	1	2554101496	0.183229	0.178202	0.432887	0	0.949434	Low
...	...	...	...	...	...	...	...	...	...	...	...
86499	3856	0	798993758	1	799084113	0.000015	0.000124	0.235934	1	6465.933565	High
42809	580	0	24364820	1	24408209	0.000007	0.000019	0.007195	2	398.410334	High
19939648	141191	1	107544414	1	127625253	0.003375	0.004548	0.031757	1	3.777572	High
391480	6817	1	1432119	1	1830416	0.000066	0.000220	0.000423	1	2.565976	Medium
14271313	208126	1	62580633	1	77060072	0.002415	0.006703	0.018479	2	3.073920	Medium

Engagement Score Formula

$$\frac{(\text{Likes} + \text{Streams})}{\text{Views}}$$

Classification was used to group songs into categories based on their engagement level. This helps capture engagement quality rather than raw popularity.

Categories:

Low Engagement

Medium Engagement

High Engagement

Future Work & Conclusion

Limitations

- Used only a subset of the dataset → may reduce accuracy
- Missing important features (song duration, artist popularity, listener data)
- Limited insights due to incomplete data

Future Improvements

- Apply advanced models (multiple regression, decision trees)
- Use PCA for better data reduction
- Apply Elbow Method for optimal clustering
- Include more features and better visualizations

Conclusion

This project demonstrated how **preprocessing and ML models work together** to extract meaningful insights from real-world data.

